Registro de uso de Inteligencia Artificial

Herramienta: Claude Code (modelo Claude Opus 5.5, extensión de VS Code) **Equipo:** [nombres del equipo] · **Fecha:** 4-nov-2026 **Registro automático:** cada prompt quedó guardado en `registro-ia/prompts.md` y cada archivo tocado en `registro-ia/archivos.md`.

Principios que aplicamos (Marco Ético para la IA en Colombia, 2021)

- **Supervisión humana:** el equipo revisó, probó y entiende todo el código entregado; las decisiones (stack, nube, base de datos) las tomó el equipo.
- **Transparencia:** cada uso de IA está registrado aquí, con su prompt.
- **Privacidad:** no compartimos con la IA datos personales reales ni la cadena de conexión de la BD (se pegó directamente en `.env`).
- **No discriminación:** los datos de riesgo son de demostración y describen factores (luz, soledad, reportes), no barrios.
- **Responsabilidad:** el equipo responde por la calidad, originalidad y funcionamiento.

Entradas

Fase 0 — Andamiaje

- **Prompt / instrucción relevante:** "leee claude.md" → la IA leyó el `CLAUDE.md` del equipo y propuso un plan ajustado a la hora real (09:07). Respuesta del equipo: "en azure, y dale".
- **Qué generó la IA:** estructura MVC, `package.json`, `app.js` con helmet (CSP), CORS restringido, cookie-parser, manejo central de errores, `.env.example` y `.gitignore`.
- **Qué revisó o corrigió el equipo:** eligió desplegar en **Azure** (no en Render, como decía el `CLAUDE.md`) porque el reglamento del ciclo pide AWS, GCP o Azure. Se bloqueó un comando de la IA que cerraba *todos* los procesos `node` del PC; se cambió por detener solo el servidor de prueba.
- **Evaluación crítica:** la IA propuso `bcryptjs` en lugar de `bcrypt` para evitar compilar código nativo en Windows/Node 24; es el mismo algoritmo, pero más lento. Lo aceptamos y lo documentamos.
- **Valor aportado:** ~30 min ahorrados en configuración repetitiva.

Fase 1 — Base de datos y autenticación

- **Prompt / instrucción relevante:** "un momento necesito instalar PostgreSQL y qué es ese neon.tech?" → la IA explicó que Neon es PostgreSQL administrado en la nube (capa gratuita) y que no hacía falta instalar nada.
- **Qué generó la IA:** `db/schema.sql` (13 tablas, con `CHECK` de valores), `db/seed.sql` con datos **demo** marcados, `db/init.js` que hashea las contraseñas de prueba, modelos `User` / `City`, `authService` (bcrypt costo 11, JWT HS256 de 2 h en cookie httpOnly + sameSite strict), validaciones con express-validator, rate limit en login/registro y vistas accesibles de ingreso y registro.
- **Qué revisó o corrigió el equipo:** decidió no pegar la URL de la BD en el chat. Neon sugería instalar su CLI (`neon init` / `neon deploy`); se descartó porque bastaba con la URL en `.env` y el driver `pg`. Se cambió `sslmode=require` por `verify-full` para validar el certificado del servidor.
- **Pruebas hechas:** contraseña débil → 400; registro → cookie `HttpOnly` y redirección al onboarding; `/map` sin sesión → `/login`; login fallido → mensaje genérico que no revela si el correo existe.
- **Evaluación crítica:** el rol va dentro del JWT; si un admin pierde el rol, el cambio se aplica cuando vence el token (2 h). Es un riesgo aceptado y documentado.

Fase 2 — Mapa y capas (datos abiertos)

- **Prompt / instrucción relevante:** "dale" (continuar con el plan del `CLAUDE.md`, sección 10).
- **Qué generó la IA:** `GET /api/map/layers`, modelos `RiskZone` / `SafePoint`, `overpassService` (OpenStreetMap vía Overpass) con script `npm run osm:sync`, mapa Leaflet servido desde `node_modules` (sin CDN) y lista textual de las capas como alternativa accesible al mapa.
- **Qué revisó o corrigió el equipo:** la primera consulta a Overpass falló por tiempo de espera. Se agregó un servidor espejo y en el segundo intento llegaron **datos reales**: 5 estaciones de policía, 16 luminarias y 7 lugares abiertos 24 h en Girardot.
- **Evaluación crítica:** OSM tiene poca cobertura de luminarias en Girardot (16). El puntaje depende de la calidad de esos datos; por eso se combinan con reportes moderados de la comunidad.

Fase 3 — Ruteo y puntaje de seguridad (núcleo)

- **Prompt / instrucción relevante:** sección 6 del `CLAUDE.md` (fórmula del puntaje).
- **Qué generó la IA:** `geoUtils` (haversine, muestreo cada 30 m, celdas de ~100 m), `safetyScoreService` con funciones puras y pesos en `WEIGHTS`, `routingService` (ORS → OSRM peatonal de respaldo), `routePlanner` y búsqueda de lugares con Nominatim. También 11 pruebas unitarias con `node:test`.
- **Qué revisó o corrigió el equipo:**
  1. **Error detectado:** la IA advirtió que el OSRM público (`router.project-osrm.org`) solo calcula rutas en **carro** aunque se pida "foot". Se usó el servidor OSRM de FOSSGIS con perfil peatonal.
  2. **Desviación del `CLAUDE.md`:** la fórmula decía "normalizado por número de puntos". Eso diluye una zona peligrosa en rutas largas, así que se decidió penalizar **por metro expuesto**: 0,6 × nivel por cada tramo de 30 m.
  3. **Mejora:** como las alternativas de la API casi siempre pasaban por las mismas zonas, se agregaron **rutas de desvío** con puntos intermedios a cada lado de la zona más grave.
  4. Se corrigió la redacción de la explicación ("unos 1 min" → "1 min").
- **Evaluación crítica (sesgos):** la explicación describe **factores** (luz, soledad, reportes) y nunca nombra barrios, para no estigmatizar sectores. El puntaje es una estimación con datos demo y no una garantía; la interfaz lo dice en cada pantalla.

Fase 4 — Recorrido, historial, calificación y emergencia

- **Qué generó la IA:** modelos `Trip`, `TripRating`, `SegmentScore`, `Alert` y `EmergencyContact`; cifrado AES-256-GCM del teléfono y el correo de los contactos (`cryptoService`); botón de ayuda con 3 s para cancelar, `tel:123` y enlaces de WhatsApp con el mensaje ya escrito; alarma con Web Audio API; Screen Wake Lock.
- **Qué revisó o corrigió el equipo:** sin credenciales de Telegram el aviso se **simula** y así se le informa a la usuaria ("envío simulado en esta demo"), en vez de fingir que se envió.
- **Pruebas hechas:** ciclo trazar → iniciar → alerta → finalizar → calificar 5★ → historial; el puntaje de la misma ruta subió de 74 a 84 por la retroalimentación. Pedir un recorrido de otra usuaria (IDOR) responde 404. En la BD el teléfono se ve cifrado (`haYuiz…`).
- **Evaluación crítica:** una página web no puede subir el volumen del celular ni seguir funcionando con garantía en segundo plano; se documenta en el onboarding y en el recorrido.

Fase 6 (parcial) — Control de inactividad

- **Motivo:** la pantalla "Mi red" ofrecía configurar la inactividad y la IA detectó que esa función aún no existía. Se implementó para que la interfaz **no prometa algo que no hace**.
- **Qué generó la IA:** temporizador en `trip.js` que se reinicia con cada toque, diálogo "¿Estás bien?" con 20 s, alarma (el audio se "arma" con el primer toque porque el navegador lo exige) y alerta automática con `reason = 'inactividad'`.
- **Evaluación crítica:** si el navegador queda en segundo plano, el temporizador puede pausarse. Queda documentado como limitación.

Fase 8 — Despliegue

- **Prompts relevantes:** "al momento de crear la app en azure me apareció esto", "cuál de estos selecciono", "me aparece ahora esto" (con capturas de los errores).
- **Qué hizo la IA:** guió la creación en Azure paso a paso. Interpretó cada error: primero la política de regiones de Azure for Students, luego la **cuota 0** de F1 y de B1. Propuso Render como plan B, preparó el commit y el push (verificando que `.env` no se subiera) y generó un archivo local con las variables de entorno para no copiarlas a mano.
- **Qué decidió el equipo:** intentar Azure primero (lo pide el reglamento), revisar la política de la suscripción sin modificarla, crear un repositorio **privado** en GitHub y aceptar Render como alternativa declarada.
- **Pruebas en producción:** login por API, cookie `Secure` + `httpOnly`, cabeceras HSTS y CSP, capas, cálculo de ruta real (OSRM) y todas las pantallas con 200.
- **Evaluación crítica:** la IA supuso al principio que la región "East US 2" estaba permitida y tuvo que corregirse con la lista real de la política. Los errores de cuota de Azure no se podían resolver en el tiempo disponible; se documentaron con honestidad en lugar de ocultarlos.

Fase 7 (adelantada) — Reportes y moderación (segundo rol)

- **Motivo:** el reglamento exige al menos 2 roles con permisos distintos.

- **Qué generó la IA:** reporte anónimo de problemas desde el mapa (`POST /api/incidents`, estado `pending`) y panel `/admin/incidents` donde la administradora aprueba (crea una zona `comunidad`) o rechaza.
- **Pruebas hechas:** la usuaria recibe 403 en moderación; la admin aprueba y aparece la zona; un `<script>` en la descripción se muestra escapado (XSS mitigado).
- **Evaluación crítica:** la moderación humana evita que un reporte malicioso o difamatorio marque un sector como peligroso.

Resumen de herramientas

Herramienta	Para qué	Prompt relevante	Resultado	Qué corregimos o rechazamos
Claude Code (Opus 5.5)	Plan, código MVC, pruebas, documentación y guía de despliegue	"leee claude.md" + "dale"	App completa con 11 pruebas unitarias	Fórmula del puntaje, OSRM en carro → peatonal, comando destructivo bloqueado, Azure → Render
Claude Code	Explicar Neon y PostgreSQL en la nube	"qué es ese neon.tech?"	BD en la nube sin instalar nada	Descartamos instalar la CLI de Neon; la URL no se pegó en el chat
Claude Code	Interpretar errores de Azure	Capturas de los errores	Diagnóstico de política de regiones y cuota 0	No modificamos la política de la cuenta; usamos el plan B
[Otras IA que haya usado el equipo]	[completar]	[completar]	[completar]	[completar]

Marco Ético para la IA en Colombia (2021): cómo lo aplicamos

Principio	Cómo se aplicó
Transparencia y explicación	Todo uso de IA está aquí y en `registro-ia/`. La app explica **por qué** recomienda una ruta y cómo se calcula el puntaje.
Privacidad	Consentimiento explícito de ubicación, contactos cifrados, borrado de cuenta y ningún dato real ni secreto compartido con la IA.
Control humano de las decisiones	La IA propone y el equipo decide (nube, BD, fórmula). En la app, la usuaria elige la ruta y una persona modera cada reporte.
Seguridad	OWASP Top 10 aplicado y probado: rate limit, CSP, IDOR, XSS, cifrado y HTTPS.
Responsabilidad	El equipo revisó y probó cada fase y responde por el código entregado.
No discriminación	La explicación de la ruta describe factores y nunca barrios; los datos de riesgo son demo y declarados como tales.
Inclusión	WCAG 2.1 AA, tipografía para baja visión, uso con una mano, modo noche y alternativas textuales al mapa.
Prevalencia de los derechos de niñas, niños y adolescentes	La app es para mujeres adultas y no recoge datos de menores. Una versión para adolescentes exigiría consentimiento de acudientes.
Beneficio social	Atiende el ODS 5 y el ODS 11: moverse de noche con más información y acompañamiento, a costo $0 y replicable en otros municipios.

Reflexión final

- **Dónde la IA ayudó más:** acelerar el andamiaje MVC, la seguridad repetitiva (validaciones, cabeceras, cifrado) y la documentación. También detectó errores que no habríamos visto a tiempo, como que el OSRM público solo calcula rutas en carro o que una pantalla prometía una función inexistente.
- **Dónde ayudó menos:** supuso regiones de Azure sin conocer la política real de la cuenta. La configuración externa (Azure, GitHub, Render) siempre necesitó verificación humana.
- **Riesgos éticos de este proyecto y cómo los mitigamos:**
  - *Estigmatización de zonas:* la explicación describe factores, hay moderación humana y los datos son demo declarados.
  - *Privacidad de la ubicación:* consentimiento explícito, sin rastreo continuo guardado y borrado de cuenta disponible.

- *Falsa sensación de seguridad:* el aviso "apoyo, no garantía; en emergencia llama al 123" aparece en todas las pantallas clave.